GymFlow · Guion de presentación

20 de octubre de 2026 · 20 minutos · 4 minutos por integrante

Las notas también están dentro del PowerPoint. Las diapositivas 21–24 son anexos ocultos en el pase principal. Preparar y ensayar la demo local antes de la exposición.

DIAPOSITIVA 1: Menos gestión. Más movimiento.
PONENTE: Miguel — Product Owner
TRAMO: 00:00–00:30 | DURACIÓN: 30 s

GUION SUGERIDO
Somos el equipo de GymFlow. Nuestro objetivo es que un gimnasio pueda dedicar menos tiempo a reconciliar registros y más tiempo a atender a sus socios. Vamos a recorrer primero la necesidad del negocio y el flujo del cliente; después veremos cómo nos organizamos y cómo lo hemos construido.

TRANSICIÓN
Empezamos por el problema que queremos resolver.

FUENTES
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/docs/PRD.md
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/README.md

DIAPOSITIVA 2: Una operación conectada, sin registros dispersos
PONENTE: Miguel — Product Owner
TRAMO: 00:30–01:25 | DURACIÓN: 55 s

GUION SUGERIDO

Partimos de una necesidad concreta: cuando clientes, pagos y reservas viven en hojas distintas, la información se vuelve difícil de coordinar. El problema no es únicamente guardar datos. Es tomar decisiones consistentes: quién puede reservar, qué clase tiene plazas y qué ocurre si una persona cancela. GymFlow centraliza estas operaciones y ofrece una interfaz para que el socio consulte y gestione lo suyo. El valor que perseguimos es reducir errores operativos y facilitar la atención. No presentamos porcentajes de ahorro medidos; mostramos una solución funcional que permite comprobar estos comportamientos.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/docs/PRD.md
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/docs/client/CLIENT_SPECS.md

DIAPOSITIVA 3: Tres perfiles, una experiencia coordinada
PONENTE: Miguel — Product Owner
TRAMO: 01:25–02:20 | DURACIÓN: 55 s

GUION SUGERIDO

La solución contempla tres perfiles. El cliente quiere saber qué clases hay, reservar una plaza y consultar su información sin depender siempre de recepción. El entrenador necesita trabajar con sus propias clases y sus reservas. La administración necesita una visión más amplia para gestionar la operación y exportar información. Esta distinción también guía los permisos: el entrenador no administra todo el gimnasio y un cliente no consulta datos de otros socios. Además, tener una cuenta para acceder no equivale a tener una membresía activa. Esa separación evita confundir identidad con las reglas del negocio.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/AGENTS.md
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/docs/adr/0001-cuenta-separada-de-cliente.md

DIAPOSITIVA 4: Del acceso a una plaza confirmada
PONENTE: Miguel — Product Owner
TRAMO: 02:20–04:00 | DURACIÓN: 100 s

GUION SUGERIDO

El recorrido principal es sencillo: el socio accede, elige una clase, consulta disponibilidad y reserva. La aplicación confirma esa reserva si se cumplen las reglas del gimnasio: membresía activa, al menos un pago registrado, plaza libre y ausencia de una reserva duplicada. Vamos a verlo desde el punto de vista de quien usa el producto. En el registro se puede crear un perfil de cliente, pero nace inactivo; para no dedicar la demo a preparar datos utilizaremos una cuenta de demostración ya habilitada. Al finalizar, la reserva aparece en Mis reservas. Brian retomará este recorrido para explicar la interacción con la API y la cancelación.

ACCIÓN / DEMO

Dedicar unos 40 s a explicar y 60 s a la demo. Preferencia: http://127.0.0.1:4200. Iniciar sesión con demo@gymflow.com, contraseña demo documentada Demo56789012. Usar datos ficticios. Abrir Clases, seleccionar una clase futura con plaza libre y sin reserva de esa cuenta, pulsar Reservar y mostrar Mis reservas. No cambiar credenciales ni editar datos ajenos. Si la demo se demora más de 15 s, continuar con la captura. No iniciar registro nuevo durante esta demostración.

TRANSICIÓN

Juan Carlos explica ahora cómo convertimos este flujo en trabajo coordinado.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/README.md
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/src/application/use_cases/gym_use_cases.py

DIAPOSITIVA 5: Cinco responsabilidades, una entrega compartida
PONENTE: Juan Carlos — Scrum Master
TRAMO: 04:00–04:40 | DURACIÓN: 40 s

GUION SUGERIDO

La presentación refleja nuestras cinco responsabilidades y da cuatro minutos a cada integrante. Miguel conduce la visión de negocio. Yo explico la coordinación, la organización de historias y los criterios de entrega. Fabiana presenta la lógica del backend; Brian, la experiencia de uso; y David, cómo ejecutamos y desplegamos la aplicación. La responsabilidad sobre el resultado es compartida. En las preguntas finales responde primero la persona del área y el resto complementa cuando corresponda.

FUENTES

Reparto de roles y tiempos confirmado por el equipo para el 20/10/2026.

DIAPOSITIVA 6: De una necesidad a una historia verificable
PONENTE: Juan Carlos — Scrum Master
TRAMO: 04:40–05:40 | DURACIÓN: 60 s

GUION SUGERIDO

Una historia útil describe una necesidad de una persona, no solo una tarea técnica. Este ejemplo resume el objetivo de reservar una clase. Para desarrollarlo necesitamos criterios observables: acceso válido, cliente activo, un pago registrado, disponibilidad y una reserva que pertenezca al propio cliente. Así podemos dividir el trabajo entre base de datos, reglas de la API, interfaz y pruebas sin perder el objetivo común. En GitHub distinguimos historias y tareas, y cada tarea mantiene responsable, prioridad, dependencias y criterios de aceptación. Las condiciones que vemos aquí están respaldadas por el comportamiento y los tests del proyecto.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/AGENTS.md
https://github.com/IA-P1-BCN/g1-python-crud-api/issues/36
https://github.com/IA-P1-BCN/g1-python-crud-api/issues/38
https://github.com/IA-P1-BCN/g1-python-crud-api/issues/106

DIAPOSITIVA 7: Priorizar y desbloquear antes de desarrollar
PONENTE: Juan Carlos — Scrum Master
TRAMO: 05:40–06:50 | DURACIÓN: 70 s

GUION SUGERIDO

El tablero hace visible qué está pendiente, qué está preparado, qué está en curso y qué necesita revisión. La coordinación también consiste en identificar el orden correcto. Por ejemplo, primero protegimos los endpoints con JWT en la tarea 105; sobre esa base, la 106 añadió límites por rol y propietario. Empezar por la segunda sin resolver la primera habría creado una dependencia incompleta. Los milestones organizan niveles de entrega, y las etiquetas y campos de prioridad ayudan a seleccionar trabajo. Desde Scrum, el foco es mantener un objetivo compartido, revisar avances y desbloquear al equipo. La evidencia que mostramos son las historias, las dependencias y el tablero; no atribuimos una cadencia de ceremonias que no esté documentada.

ACCIÓN / DEMO

Se puede abrir el tablero durante 10–15 s si la sesión de GitHub ya está iniciada. No navegar buscando tareas durante la exposición. Seleccionar previamente #105 y #106.

FUENTES

https://github.com/orgs/IA-P1-BCN/projects/32/views/1
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/AGENTS.md
https://github.com/IA-P1-BCN/g1-python-crud-api/issues/105
https://github.com/IA-P1-BCN/g1-python-crud-api/issues/106

DIAPOSITIVA 8: Una tarea termina cuando se puede comprobar
PONENTE: Juan Carlos — Scrum Master
TRAMO: 06:50–08:00 | DURACIÓN: 70 s

GUION SUGERIDO

Nuestra definición de terminado no es simplemente que una pantalla funcione una vez. Antes de desarrollar registramos requisitos y escenarios en OpenSpec. Después implementamos respetando las capas y las reglas del proyecto. Verificamos estilo, tests y la estructura de la especificación. Finalmente, una pull request relaciona el cambio con su issue y necesita revisión humana para entrar en main. El uso de IA también queda documentado: prompts relevantes, decisiones, errores y correcciones. La IA acelera la ejecución, pero la responsabilidad del diseño y del código sigue siendo del equipo. Una lección concreta fue aclarar el alcance del entrenador antes de implementar sus permisos.

TRANSICIÓN

Fabiana nos muestra cómo esas decisiones se traducen en reglas y arquitectura del backend.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/docs/SPEC_WORKFLOW.md
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/docs/ai-log.md
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/.github/pull_request_template.md
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/AGENTS.md

DIAPOSITIVA 9: El negocio no depende del framework
PONENTE: Fabiana — Backend
TRAMO: 08:00–09:05 | DURACIÓN: 65 s

GUION SUGERIDO

El backend usa Python y FastAPI, pero las reglas del gimnasio no viven mezcladas con el framework. Interfaces recibe HTTP, valida con schemas y transforma respuestas. Application contiene los casos de uso. Domain define entidades con dataclasses y contratos con Protocol. Infrastructure implementa esos contratos con SQLAlchemy, MySQL y los adaptadores de seguridad. La dirección de dependencias apunta hacia el dominio; application no importa infraestructura. El wiring de dependencias conecta las piezas. Combinamos orientación a objetos, casos de uso, repositorios e inyección de dependencias. Esto permite probar reglas con un repositorio en memoria y confirmar después su funcionamiento con HTTP y MySQL reales.

ACCIÓN / DEMO

Mostrar brevemente en el editor src/application/use_cases/gym_use_cases.py y src/domain/repositories/gym_port.py. Tener ambos archivos abiertos antes del turno.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/AGENTS.md
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/src/interfaces/http/dependencies.py
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/src/domain/repositories/gym_port.py

DIAPOSITIVA 10: Una identidad; dos vínculos de negocio
PONENTE: Fabiana — Backend
TRAMO: 09:05–10:00 | DURACIÓN: 55 s

GUION SUGERIDO

El modelo separa seis entidades de negocio y acceso, más una tabla técnica de versiones. Un cliente tiene pagos y reservas. Un entrenador imparte clases. La relación de muchos a muchos entre clientes y clases se resuelve con Reservas, que guarda además asistencia y fecha. Cuenta es una identidad separada, con vínculos opcionales y únicos al cliente o al entrenador. Así evitamos duplicar credenciales y crear socios ficticios para el personal. Las claves foráneas protegen la consistencia y RESTRICT impide borrar registros que mantienen dependencias. La combinación cliente y clase es única. Alembic evoluciona el esquema sin recrear toda la base.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/docs/EDR_DIAGRAM.md
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/docs/DATABASE_SPEC.md
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/docs/adr/0001-cuenta-separada-de-cliente.md

DIAPOSITIVA 11: Autenticar no basta: hay que delimitar el acceso
PONENTE: Fabiana — Backend
TRAMO: 10:00–10:50 | DURACIÓN: 50 s

GUION SUGERIDO

El login comprueba una contraseña almacenada como hash Argon2id y emite un JWT con caducidad configurable. Después, cada petición protegida verifica el token y vuelve a leer la cuenta para comprobar que sigue activa y conocer su rol y vínculo actuales. Admin gestiona todo. Staff solo opera sobre las clases de su entrenador vinculado y las reservas de esas clases. Cliente solo consulta sus datos y reserva para sí mismo. Los listados aplican ese alcance antes de paginar. Sin token válido respondemos 401; una operación prohibida da 403, y los detalles ajenos de recursos permitidos dan 404 sin revelar datos.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/src/application/use_cases/auth_use_cases.py
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/src/application/use_cases/gym_use_cases.py
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/src/infrastructure/security/tokens.py

DIAPOSITIVA 12: La última plaza se decide en una transacción
PONENTE: Fabiana — Backend
TRAMO: 10:50–12:00 | DURACIÓN: 70 s

GUION SUGERIDO

Esta es una regla de negocio que merece protección en el servidor. Para una nueva reserva o un traslado, se verifica el cliente, se bloquea la clase y se comprueban pago, unicidad y aforo dentro de la transacción. El fragmento muestra el bloqueo y dos condiciones; hay líneas intermedias omitidas para que resulte legible. En el repositorio, lock=True se traduce a SELECT FOR UPDATE mediante SQLAlchemy. Si dos personas intentan ocupar la última plaza, las admisiones se serializan: no se crean dos reservas sobre el mismo cupo. Un conflicto devuelve 409 y revierte la operación. Cancelar libera la plaza. En los nombres seguimos snake_case para funciones y variables, PascalCase para clases y constantes en mayúsculas.

ACCIÓN / DEMO

Durante 20–25 s, mostrar el bloque completo en src/application/use_cases/gym_use_cases.py y el método get de src/infrastructure/repositories/gym_repository.py. Señalar with_for_update(). No ejecutar cambios en código durante la presentación.

TRANSICIÓN

Brian explica cómo esta lógica se convierte en una experiencia clara para el usuario.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/src/application/use_cases/gym_use_cases.py
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/src/infrastructure/repositories/gym_repository.py
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/tests/integration/test_gym_crud.py

DIAPOSITIVA 13: Una interfaz que hace visible el estado del gimnasio
PONENTE: Brian — Frontend
TRAMO: 12:00–13:00 | DURACIÓN: 60 s

GUION SUGERIDO

El frontend está construido con Angular 22 y TypeScript, utilizando Tailwind y componentes de Spartan. Organizamos la experiencia por funcionalidades: acceso, registro, clases, reservas, perfil y pantallas de gestión. Los componentes compartidos resuelven piezas repetidas, como las tarjetas de clase, el indicador de cupo, los avisos y las confirmaciones. Los formularios validan entradas y muestran mensajes relacionados con el problema. La sesión se mantiene en la pestaña con sessionStorage y signals representan su estado en la interfaz. Las capturas son de la aplicación real, conectada al backend local; no son diseños conceptuales.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/frontend/package.json
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/frontend/src/app/core/auth/session.service.ts
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/frontend/src/app/shared/components
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/frontend/src/app/features/perfil/pages/perfil.page.ts

DIAPOSITIVA 14: De la pantalla a la API, con un contrato compartido
PONENTE: Brian — Frontend
TRAMO: 13:00–14:00 | DURACIÓN: 60 s

GUION SUGERIDO

La página no conoce los detalles de persistencia. Delega en un servicio que utiliza contratos tipados y llama a la API. El interceptor añade el JWT a las peticiones de /v1. Si el servidor devuelve 401 para una sesión existente, se borra el token y se cierra la sesión local; el caso de credenciales incorrectas en el login se trata por separado. Otro detalle importante es el cupo: un cliente solo puede ver sus reservas, por eso no calculamos disponibilidad contando esa lista. Consultamos el agregado de la clase y mostramos plazas libres sin recibir identidades ajenas. En TypeScript utilizamos camelCase en funciones y propiedades, y PascalCase en clases e interfaces.

ACCIÓN / DEMO

Mostrar durante unos segundos frontend/src/app/core/http/auth.interceptor.ts y frontend/src/app/services/clases-con-cupo.ts. El extracto en la diapositiva conserva la lógica original con saltos de línea adaptados.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/frontend/src/app/core/http/auth.interceptor.ts
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/frontend/src/app/services/clases-con-cupo.ts
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/frontend/src/app/services/api/recursos.api.ts

DIAPOSITIVA 15: La demo termina donde el usuario espera
PONENTE: Brian — Frontend
TRAMO: 14:00–15:25 | DURACIÓN: 85 s

GUION SUGERIDO

Retomamos la experiencia de Miguel. La reserva aparece en el espacio personal y el usuario puede cancelarla mediante una confirmación. Después de cancelar, la lista se actualiza y la clase vuelve a disponer de esa plaza. Esta coherencia se mantiene porque la interfaz consulta el resultado real del servidor. No mostramos un éxito si la API ha rechazado la operación. Si el tiempo lo permite, señalaré la petición en Network: POST para crear una reserva, GET para consultarla y DELETE para cancelarla. Lo importante para el usuario es que la acción tenga una respuesta clara y que el estado que ve coincida con la operación del gimnasio.

ACCIÓN / DEMO

Reservar 65–70 s para la demo y 15–20 s para explicación. Continuar con la reserva de Miguel; si no se creó, utilizar una clase ya preparada. Abrir Mis reservas → Cancelar → confirmar → volver a Clases y comprobar disponibilidad. Opcional: Network filtrado por /v1/reservas, sin mostrar cabeceras Authorization ni tokens. Si la conexión falla, explicar con la captura y continuar al minuto 15:25.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/frontend/src/app/features/reservas/pages/mis-reservas.page.ts
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/frontend/src/app/services/http-reserva.service.ts

DIAPOSITIVA 16: Los errores también forman parte de la experiencia
PONENTE: Brian — Frontend
TRAMO: 15:25–16:00 | DURACIÓN: 35 s

GUION SUGERIDO

Una experiencia completa también explica por qué algo no se ha podido hacer. Un 401 pide recuperar el acceso; un 409 informa de un conflicto, como falta de plazas; un 422 ayuda a corregir datos. Los fallos de red se presentan con un mensaje comprensible. La suite del frontend comprueba formularios, sesión, componentes e integración HTTP simulada: son 117 pruebas en 26 suites, verificadas hoy. La autorización real permanece siempre en el backend.

TRANSICIÓN

David cierra con la ejecución, el despliegue y la evidencia de calidad.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/frontend/src/app/core/http/peticion.ts

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/frontend/src/app/services/http-reserva.service.spec.ts

npm test -- --watch=false · 19/10/2026 · 117 passed / 26 suites

DIAPOSITIVA 17: Una aplicación desplegable, con datos persistentes
PONENTE: David — DevOps
TRAMO: 16:00–17:10 | DURACIÓN: 70 s

GUION SUGERIDO

La arquitectura de despliegue está definida en el repositorio. Un contenedor en Render sirve FastAPI y también la compilación de Angular, de forma que frontend y API comparten origen. La base MySQL es un servicio externo gestionado en Aiven, y la conexión utiliza TLS. En local, Docker Compose coordina tres servicios: MySQL, API y frontend servido por Nginx. El Dockerfile utiliza varias etapas: compila Angular, instala dependencias Python con uv y prepara una imagen de ejecución con un usuario sin privilegios. Las variables sensibles se configuran fuera del código. Para la demo técnica hemos preparado el entorno local con el esquema actual; hay que comprobar la paridad del despliegue público antes de usarlo en directo.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/Dockerfile
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/docker-compose.yml
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/render.yaml
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/config/settings.py
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/README.md

DIAPOSITIVA 18: Del pull request a una entrega repetible
PONENTE: David — DevOps
TRAMO: 17:10–18:10 | DURACIÓN: 60 s

GUION SUGERIDO

El repositorio utiliza ramas feature por tema, commits convencionales en español y pull requests a main con referencia a la issue. GitHub Actions ejecuta Ruff y pytest con una instancia real de MySQL y migraciones. La revisión humana es parte del flujo antes de integrar. El despliegue está configurado en Render para reaccionar a commits de main: no es un job de despliegue dentro de GitHub Actions. Conviene distinguir ambas funciones. El arranque del contenedor aplica migraciones, ejecuta el seed idempotente y arranca Uvicorn. Eso reduce pasos manuales y mantiene el proceso de ejecución descrito y reproducible.

ACCIÓN / DEMO

El YAML se muestra con el valor commit en la línea siguiente por legibilidad; en render.yaml figura autoDeployTrigger: commit. Si se abre código, usar el fichero real. Abrir la ejecución de CI ya preparada; no mostrar secretos del dashboard.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/.github/workflows/tests.yml
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/render.yaml
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/scripts/docker-entrypoint.sh
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/AGENTS.md

DIAPOSITIVA 19: Calidad y documentación que se pueden abrir
PONENTE: David — DevOps
TRAMO: 18:10–19:15 | DURACIÓN: 65 s

GUION SUGERIDO

La calidad se apoya en evidencia concreta. La última ejecución de main en GitHub Actions pasó 376 pruebas de backend; incluye integración HTTP contra el servidor y MySQL, además de pruebas unitarias. En el frontend, la verificación local del 19 de octubre pasó 117 pruebas en 26 suites. Son cifras de pruebas ejecutadas, no porcentajes de cobertura. La documentación también está organizada para distintas necesidades: Swagger y ReDoc para consumir la API; README y Taskfile para ejecutar; diagrama ER y diccionario para entender los datos; y OpenSpec y la bitácora para seguir decisiones. Los healthchecks permiten distinguir si responde la aplicación y si la base de datos está disponible.

ACCIÓN / DEMO

Enlazar la cifra backend a la ejecución real: https://github.com/IA-P1-BCN/g1-python-crud-api/actions/runs/39213569324. La documentación pública puede reflejar una versión distinta; para mostrar el contrato actual usar http://127.0.0.1:8000/docs.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/actions/runs/39213569324
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/.github/workflows/tests.yml
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/docs/ai-log/2026-10-16-permisos-gimnasio-106.md
npm test -- --watch=false · 19/10/2026 · 117 passed / 26 suites

DIAPOSITIVA 20: Un gimnasio conectado. Un equipo responsable.
PONENTE: David — DevOps
TRAMO: 19:15–20:00 | DURACIÓN: 45 s

GUION SUGERIDO

Con GymFlow hemos conectado una necesidad de negocio con una solución completa: información centralizada, reservas sujetas a reglas, acceso por rol y una aplicación que podemos ejecutar y desplegar de forma reproducible. La entrega actual incluye el portal, la API, la base de datos, las pruebas y su documentación. Quedan evoluciones posibles, como servicios de pago, notificaciones y seguridad avanzada, que no presentamos como funcionalidades ya entregadas. Gracias. Para las preguntas, cada integrante responderá primero sobre su área y el resto podrá complementar.

ACCIÓN / DEMO

Cerrar antes del minuto 20. Las preguntas se atienden después del bloque cronometrado, según indique la organización. Miguel: producto; Juan Carlos: proceso; Fabiana: API y datos; Brian: interfaz; David: infraestructura.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/README.md
https://github.com/IA-P1-BCN/g1-python-crud-api

DIAPOSITIVA 21: Reloj de ensayo y relevos
PONENTE: Equipo — Apoyo
TRAMO: 20:00–20:00 | DURACIÓN: 0 s

GUION SUGERIDO

Este anexo es una herramienta de ensayo y no forma parte de los veinte minutos. Cada persona debe preparar su bloque con el cronómetro. A los tres minutos y cuarenta segundos se muestra una señal de cierre para garantizar el relevo. Mantener las pestañas y ficheros abiertos previamente. Si alguien falta, acordar antes de la presentación el reparto de su bloque entre dos compañeros y comunicarlo a la organización; no concentrar todas las respuestas en una sola persona.

ACCIÓN / DEMO

Relevos de apoyo sugeridos para ensayar: Miguel y Juan Carlos se cubren negocio/proceso; Fabiana y Brian se cubren el recorrido API/interfaz; David y Fabiana se cubren arranque y persistencia. Confirmar el reparto si hay ausencia. No activar avance automático: las demos requieren control manual.

FUENTES

Estructura y tiempos solicitados por el equipo.

Anexo oculto en el pase principal; abrir solo como apoyo.

DIAPOSITIVA 22: Preparar una demo que no dependa de improvisar
PONENTE: Equipo — Apoyo
TRAMO: 20:00–20:00 | DURACIÓN: 0 s

GUION SUGERIDO

Preparación operativa: ejecutar task up o task docker:up con antelación, comprobar /health y /health/db, iniciar sesión con la cuenta demo y asegurarse de que hay una clase disponible. El registro nuevo puede crear un cliente inactivo, por lo que no debe sustituir a la cuenta preparada en la demo. Durante la verificación del 19/10, el entorno local respondió versión 0.4 y el público versión 0.3; por ello no se afirma paridad de producción. La presentación describe la implementación de main y propone demostrarla en local. El servidor público respondió, pero necesitó tiempo de espera. Las capturas de la presentación proceden del entorno local con datos ficticios.

ACCIÓN / DEMO

URL local: http://127.0.0.1:4200. Swagger local: http://127.0.0.1:8000/docs. Cuenta demo: demo@gymflow.com / Demo56789012, credenciales de demostración documentadas, nunca las del administrador. Ensayar reservar y cancelar antes del acto, sin dejar una reserva duplicada. Si falla la conexión durante el acto, volver a las capturas incluidas y mantener el reloj.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/README.md
Verificación HTTP del 19/10/2026: local /health/db 200, versión 0.4; Render /health/db 200, versión 0.3.

Anexo oculto en el pase principal; abrir solo como apoyo.

DIAPOSITIVA 23: Distinguir la entrega actual de las siguientes mejoras
PONENTE: Equipo — Apoyo
TRAMO: 20:00–20:00 | DURACIÓN: 0 s

GUION SUGERIDO

La columna izquierda resume lo implementado. La derecha agrupa capacidades que aparecen en el plan y no se presentan como terminadas. Registrar un pago no equivale a procesar una transacción de tarjeta mediante una pasarela. Asimismo, OAuth2 Password Flow con JWT no es un login social. La autorización de reservas comprueba que existe al menos un pago, pero no calcula vencimiento porque no hay periodos de membresía en el modelo actual. Estas distinciones permiten responder con precisión sobre el alcance.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/AGENTS.md
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/docs/client/CLIENT_SPECS_V2_USER_MNG.md
https://github.com/IA-P1-BCN/g1-python-crud-api/issues/39
https://github.com/IA-P1-BCN/g1-python-crud-api/issues/41
https://github.com/IA-P1-BCN/g1-python-crud-api/issues/50
https://github.com/IA-P1-BCN/g1-python-crud-api/issues/52

Anexo oculto en el pase principal; abrir solo como apoyo.

DIAPOSITIVA 24: Fuentes verificables y preguntas repartidas
PONENTE: Equipo — Apoyo
TRAMO: 20:00–20:00 | DURACIÓN: 0 s

GUION SUGERIDO

Todos los datos de la presentación se apoyan en el repositorio, la especificación, la ejecución de CI y las comprobaciones locales. La presentación no afirma porcentajes de cobertura ni mejoras de negocio cuantificadas. Las URLs están enlazadas para consulta. La distribución de preguntas ayuda a que los cinco integrantes participen en la defensa. Si una pregunta cruza áreas, la persona que empieza puede ceder la parte correspondiente a un compañero.

FUENTES

https://github.com/IA-P1-BCN/g1-python-crud-api
https://github.com/orgs/IA-P1-BCN/projects/32/views/1
https://github.com/IA-P1-BCN/g1-python-crud-api/actions/runs/39213569324
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/README.md
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/AGENTS.md
https://github.com/IA-P1-BCN/g1-python-crud-api/blob/2beb0b405d5a8fed009a33ad476d74dd941b217b/docs/EDR_DIAGRAM.md

Anexo oculto en el pase principal; abrir solo como apoyo.